บทที่ 2 — ใส่มาทำไม ตรงกับงานเราไหม

เอกสารนี้ไม่ใช่ส่วนของรูปเล่ม ไล่อธิบายทุกหัวข้อว่ามีอยู่ทำไมและจะเอาไปใช้ทำอะไร

หัวข้อ 2.1 เขียนจากมุมก่อนลงมือพัฒนา คือยืนอยู่บนปัญหาในบทที่ 1 อย่างเดียว ไม่อ้างถึงระบบที่สร้างเสร็จแล้ว เพราะบทที่ 2 ต้องเขียนได้ตั้งแต่ยังไม่เริ่มเขียนโค้ด แต่ละหัวข้อจึงตอบว่า **ใส่มาทำไม · ตัวอย่างให้เห็นภาพ · เราจะเอามาทำอะไร · ถ้าไม่มีหัวข้อนี้จะเป็นยังไง** โดยตัวอย่างเป็นเหตุการณ์ที่เกิดได้จริงในร้าน ใช้เล่าตอบอาจารย์ได้ทันที

ส่วน **2.2 กับ 2.3 เขียนจากมุมหลังเลือกเทคโนโลยีแล้ว** เพราะเป็นการอธิบายสิ่งที่ตัดสินใจไปแล้วว่าเลือกเพราะอะไร และมี **ตัวอย่าง** กำกับทุกหัวข้อเช่นเดียวกัน

บรรทัดสุดท้ายของทุกหัวข้อสำคัญที่สุด เพราะถ้าตอบได้ว่าไม่มีแล้วเสียอะไร แปลว่าหัวข้อนั้นมีเหตุผลอยู่จริง

2.1 ทฤษฎีที่เกี่ยวข้อง

ส่วนนี้เขียนขึ้นในขั้นตอนก่อนลงมือพัฒนา จึงไม่ได้อธิบายว่าโค้ดเขียนไว้อย่างไร แต่อธิบายว่า **ปัญหาในบทที่ 1 เรียกร้องความรู้อะไร** และ **เราตั้งใจจะเอาความรู้นั้นไปใช้ตัดสินใจเรื่องอะไร** โดยมีตัวอย่างจากหน้างานจริงประกอบทุกหัวข้อ

2.1.1 การแบ่งชั้นซอฟต์แวร์และการแยกความรับผิดชอบ

ใส่มาทำไม — บทที่ 1 หัวข้อ 1.4.1 ประกาศไว้เป็นสมมติฐานว่าระบบจะใช้สถาปัตยกรรมสามชั้น เมื่อประกาศไปแล้วก็ต้องมีที่อธิบายว่าสามชั้นคืออะไร และทำไมถึงเลือกแบบนี้แทนที่จะเขียนรวมกันเป็นก้อนเดียว

ตัวอย่างให้เห็นภาพ — สมมติเขียนรวมกันเป็นก้อนเดียว วันหนึ่งเจ้าของร้านขอให้เพิ่มปุ่มบนหน้าจอขาย ผู้พัฒนาจะต้องเข้าไปแก้ในไฟล์เดียวกับที่มีสูตรคำนวณการหักสต็อกอยู่ด้วย ความเสี่ยงคือแก้หน้าจอแล้วเผลอทำให้การตัดสต็อกพัง ทั้งที่สองเรื่องนี้ไม่เกี่ยวกันเลย · อีกตัวอย่างหนึ่งคือเครื่องพิมพ์ใบเสร็จเสียบสาย USB อยู่กับเครื่องหน้าร้าน ถ้าวันหนึ่งย้ายระบบหลังบ้านไปรันบนเครื่องอื่นหรือบนเซิร์ฟเวอร์ **ส่วนที่คุยกับเครื่องพิมพ์จะย้ายตามไปไม่ได้**

เราจะเอามาทำอะไร — ใช้เป็นเกณฑ์ตัดสินตอนวางโครงระบบว่าจะซอยระบบออกเป็นส่วนไหนบ้าง และที่สำคัญกว่าคือ **ส่วนไหนควรแยกออกไปเป็นโปรแกรมต่างหาก** เพื่อให้มีหลักคิดว่าเมื่อไรควรแยกและเมื่อไรไม่ควร ไม่ใช่แยกตามความรู้สึก

ถ้าไม่มีหัวข้อนี้ — จะอธิบายไม่ได้ว่าทำไมบางส่วนถึงต้องแยกออกไปรันต่างหาก และสมมติฐานที่ประกาศไว้ในบทที่ 1 จะไม่มีฐานรองรับ

2.1.2 ขอบเขตของธุรกรรมและคุณสมบัติ ACID

ใส่มาทำไม — ปัญหาจากหน้างานข้อ EV-07 คือยอดสินค้าคงเหลือในบันทึกไม่ตรงกับจำนวนจริง เมื่อจะแก้ด้วยระบบคอมพิวเตอร์ ต้องรู้ก่อนว่าฐานข้อมูลรับประกันความถูกต้องได้ด้วยกลไกอะไร บทที่ 1 หัวข้อ 1.4.2 จึงประกาศไว้ว่าจะใช้คุณสมบัติ ACID

ตัวอย่างให้เห็นภาพ — ตอนพนักงานคลังกดยืนยันจ่ายปูน 10 ถุง ระบบต้องทำหลายอย่างต่อกัน คือเปลี่ยนสถานะใบสั่งซื้อ หักสต็อกลง 10 บันทึกว่ามีของออก และบันทึกว่าใครเป็นคนจ่าย **สมมติทำไปได้สองอย่างแล้วไฟดับ** ผลคือ

สถานะเปลี่ยนเป็น “ส่งมอบแล้ว” แต่สต็อกยังไม่ถูกหัก ระบบจะบอกว่ายังมีปูน 50 ถุงทั้งที่ของออกจากคลังไปแล้ว 10 ถุง — และไม่มีใครรู้ว่าเกิดอะไรขึ้น

เราจะเอามาทำอะไร — ใช้ตอบคำถามออกแบบว่า **ตอนจ่ายของ อะไรบ้างที่ต้องสำเร็จไปด้วยกัน และอะไรที่แยกออกไปได้** ถ้าไม่มีหลักตัดสิน เราจะยัดทุกอย่างไว้ด้วยกันจนระบบช้า หรือแยกออกจากกันจนข้อมูลไม่สอดคล้อง

ถ้าไม่มีหัวข้อนี้ — หัวข้อ 2.1.3 จะลอย เพราะการล็อกจะมีความหมายก็ต่อเมื่ออยู่ในธุรกรรม

2.1.3 การควบคุมภาวะพร้อมกัน: การล็อกเชิงบวกและเชิงลบ

ใส่มาทำไม — ปัญหาจากหน้างานสองข้อชี้ไปที่เรื่องเดียวกัน คือ EV-06 มีการนำบิลเก่ามาใช้รับสินค้าซ้ำ และ EV-07 ยอดสต็อกคลาดเคลื่อน ทั้งสองข้อเกิดจากการที่**มีคนหลายคนทำรายการกับของชิ้นเดียวกันในเวลาใกล้กัน** ซึ่งคุณสมบัติ ACID เพียงอย่างเดียวตอบไม่ได้ เพราะ ACID รับประกันแค่ว่าธุรกรรมหนึ่งจะไม่ค้างครึ่งทาง แต่ไม่ได้ห้ามธุรกรรมสองอันเข้ามาทับกัน

ตัวอย่างให้เห็นภาพ — ช่วงเช้าที่ร้านคนเยอะ พนักงานคลังสองคนเปิดหน้าจอคิวคนละเครื่อง ทั้งคู่เห็นใบสั่งซื้อของลูกค้ารายเดียวกันที่สั่งปูน 10 ถุง ต่างคนต่างคิดว่าตัวเองจะจัดของใบนี้ แล้วกดยืนยันห่างกันไม่ถึงหนึ่งวินาที

> **ถ้าไม่มีกลไกป้องกัน** ระบบจะหักสต็อกสองครั้ง คือลดลง 20 ถุงทั้งที่ของออกจากคลังจริงแค่ 10 ถุง หรือแย่กว่านั้นคือของถูกจ่ายออกไปสองรอบให้ลูกค้าที่มารับใบเดียวกัน

จะเห็นว่าเหตุการณ์เดียวกันทำให้เกิดปัญหา **สองอย่างที่ต่างกัน** คือจ่ายซ้ำ กับสต็อกเพี้ยน และการแก้อย่างหนึ่งไม่ได้แก้อีกอย่างโดยอัตโนมัติ

เราจะเอามาทำอะไร — ใช้เลือกกลไกป้องกันตอนออกแบบขั้นตอนยืนยันจ่ายสินค้า โดยต้องตอบให้ได้ทั้งสองคำถาม คือ **สองคนกดใบเดียวกันพร้อมกัน ระบบจะตัดสินอย่างไร** และ **หักสต็อกสินค้าตัวเดียวกันพร้อมกัน ยอดจะเพี้ยนหรือไม่** จึงต้องรู้ว่ามีวิธีล็อกกี่แบบและแต่ละแบบเหมาะกับปัญหาไหน

ถ้าไม่มีหัวข้อนี้ — เสียหายมากที่สุด เพราะวัตถุประสงค์ข้อ 1.2.3 ที่ว่าจะป้องกันการรับสินค้าซ้ำ จะไม่มีทฤษฎีรองรับเลย และโครงงานจะเหลือเพียงระบบบันทึกสต็อกทั่วไป

2.1.4 เครื่องสถานะจำกัดกับวงจรชีวิตของคำสั่งซื้อ

ใส่มาทำไม — บทที่ 1 หัวข้อ 1.4.3 ประกาศไว้เต็มหัวข้อว่าจะควบคุมการรับสินค้าซ้ำด้วยการควบคุมสถานะคำสั่งซื้อ เมื่อยกเรื่องนี้เป็นสมมติฐานหลัก ก็ต้องมีทฤษฎีอธิบายว่าการควบคุมสถานะอย่างเป็นระบบทำอย่างไร

ตัวอย่างให้เห็นภาพ — ลูกค้าเลือกของครบแล้ว พนักงานขายกดสร้างรายการรอไว้ แต่ลูกค้าบอกว่าลืมกระเป๋าตังค์ แล้วเดินออกไป **ถ้าระบบบันทึกใบสั่งซื้อไว้ตั้งแต่ตอนเลือกของ** จะมีใบค้างอยู่ในระบบที่ไม่มีใครมาจ่าย พอสิ้นเดือนดูยอดขายก็จะมีใบพวกนี้ปนอยู่ ต้องมานั่งไล่ลบทีหลัง · อีกกรณีหนึ่ง สมมติผู้จัดการเผลอกดเปลี่ยนสถานะใบหนึ่งเป็น “ส่งมอบสำเร็จ” จากหน้าจอแก้ไขข้อมูล **ทั้งที่ของยังอยู่ในคลังและสต็อกยังไม่ถูกหัก** สถานะนั้นก็จะโกหกทันที

เราจะเอามาทำอะไร — ใช้ออกแบบว่าคำสั่งซื้อหนึ่งใบจะมีสถานะอะไรได้บ้าง และเปลี่ยนทางไหนได้บ้าง โดยมีคำถามที่ต้องตัดสินสองข้อ คือ **จะบันทึกใบสั่งซื้อตั้งแต่ลูกค้ายังไม่จ่ายเงินหรือไม่** และ **จะให้ตั้งค่าสถานะส่งมอบสำเร็จได้โดยตรงหรือให้เกิดจากการจ่ายของที่สำเร็จเท่านั้น**

ถ้าไม่มีหัวข้อนี้ — บทที่ 1 จะประกาศสมมติฐานไว้แต่บทที่ 2 ไม่มีอะไรรองรับ ซึ่งเห็นได้ทันทีถ้าเปิดสองบทเทียบกัน

2.1.5 บันทึกแบบเพิ่มอย่างเดียว: บัญชีการเคลื่อนไหวและร่องรอยการตรวจสอบ

ใส่มาทำไม — บทที่ 1 หัวข้อ 1.3.5 ระบุระบบบันทึกประวัติการทำรายการไว้ในขอบเขต และปัญหา EV-07 ยังทิ้งคำถามไว้ว่า **ถ้ายอดในระบบไม่ตรงกับของจริง จะรู้ได้อย่างไรว่าเริ่มเพี้ยนตั้งแต่เมื่อไร**

ตัวอย่างให้เห็นภาพ — สิ้นเดือนเจ้าของร้านให้พนักงานนับปูนในคลัง นับได้ **48 ถุง** แต่เปิดระบบดูแล้วขึ้นว่า **50 ถุง**

> **ถ้าเก็บแค่ยอดปัจจุบัน** สิ่งเดียวที่รู้คือหายไป 2 ถุง แต่ไม่รู้ว่าหายวันไหน จากรายการไหน และเกิดจากการจ่ายผิด การนับผิด หรือมีคนหยิบไปโดยไม่บันทึก
>
> **ถ้าเก็บทุกครั้งที่ยอดเปลี่ยน พร้อมยอดคงเหลือหลังเปลี่ยน** จะไล่ดูย้อนหลังได้ว่าวันที่ 12 หลังจ่ายใบนั้นเหลือ 62 ถุง วันที่ 13 หลังจ่ายอีกใบเหลือ 55 ถุง แล้วไปสะดุดตรงรายการไหนที่ตัวเลขเริ่มไม่ต่อกัน

เราจะเอามาทำอะไร — ใช้ตัดสินว่าเวลายอดสต็อกเปลี่ยน **จะเก็บอะไรไว้บ้าง** และจะเก็บอะไรเพิ่มนอกจากจำนวนที่เปลี่ยน เพื่อให้ตรวจย้อนหลังได้จริง หลักการเดียวกันยังใช้ออกแบบการบันทึกว่าใครทำอะไรกับข้อมูลไหนเมื่อไร

ถ้าไม่มีหัวข้อนี้ — จะออกแบบวิธีพิสูจน์ว่ายอดในระบบตรงกับของจริงไม่ได้ ซึ่งเป็นสิ่งที่ต้องทดสอบตามวัตถุประสงค์ข้อ 1.2.3

2.1.6 สถาปัตยกรรมขับเคลื่อนด้วยเหตุการณ์และการผลักข้อมูลผ่านช่องทางค้าง

ใส่มาทำไม — ปัญหาจากหน้างานข้อ EV-03 และ EV-04 คือหน้าร้านกับคลังอยู่คนละที่ ไม่มีช่องทางสื่อสารกัน ลูกค้าจึงต้องถือใบเสร็จเดินไปแจ้งเอง และวัตถุประสงค์ข้อ 1.2.2 กำหนดว่าจะลดเวลารอคอยโดยให้คลังเริ่มจัดของได้ก่อนลูกค้าเดินไปถึง

ตัวอย่างให้เห็นภาพ — สมมติเลือกวิธีให้หน้าจอคลังถามเซิร์ฟเวอร์ซ้ำทุก 2 วินาทีว่ามีใบใหม่ไหม ร้านเปิด 8 โมงถึง 6 โมงเย็น คือ 10 ชั่วโมง เท่ากับถาม **18,000 ครั้งต่อวันต่อหนึ่งหน้าจอ** ทั้งที่ทั้งวันมีใบสั่งซื้อจริงแค่หลักร้อยใบ แปลว่าเกือบทุกครั้งที่ถาม คำตอบคือ “ยังไม่มีอะไรใหม่” · และต่อให้ถามถี่ขนาดนั้น ก็ยังมีความหน่วงสูงสุด 2 วินาทีอยู่ดี

เราจะเอามาทำอะไร — ใช้เลือกวิธีที่จะทำให้หน้าจอที่คลังรู้เรื่องคำสั่งซื้อใหม่ได้เอง โดยเปรียบเทียบระหว่างให้หน้าจอถามซ้ำเป็นระยะ กับให้เซิร์ฟเวอร์ผลักข้อมูลมาเองเมื่อมีเหตุการณ์ ทำให้ตัดสินใจได้ด้วยเหตุผล ไม่ใช่เลือกเพราะคุ้นเคย

ถ้าไม่มีหัวข้อนี้ — วัตถุประสงค์ข้อ 1.2.2 จะไม่มีทฤษฎีรองรับ และเกณฑ์เวลาที่จะตั้งไว้จะกลายเป็นตัวเลขที่อธิบายไม่ได้ว่าทำได้อย่างไร

2.1.7 การควบคุมสิทธิ์ตามบทบาทและความมั่นคงปลอดภัยของเว็บแอปพลิเคชัน

ใส่มาทำไม — บทที่ 1 หัวข้อ 1.3.2 ระบุการควบคุมสิทธิ์ตามบทบาทไว้ในขอบเขตโดยตรง และระบบนี้มีผู้ใช้สามกลุ่มที่หน้าที่ต่างกันชัดเจน จึงต้องมีหลักว่าจะจัดการสิทธิ์อย่างไร

ตัวอย่างให้เห็นภาพ — พนักงานขายเข้าระบบแล้วไม่เห็นเมนูแดชบอร์ดของผู้จัดการ เพราะเราซ่อนเมนูไว้ ดูเผิน ๆ เหมือนปลอดภัยแล้ว **แต่ถ้าเขาเคยเห็นผู้จัดการเปิดหน้านั้นแล้วจำที่อยู่หน้าเว็บได้ แล้วพิมพ์เข้าไปตรง ๆ จะเข้าได้หรือไม่** คำตอบขึ้นอยู่กับว่าเราตรวจสิทธิ์ที่ไหน ถ้าตรวจแค่ตอนวาดเมนูก็เข้าได้ · อีกตัวอย่างคือถ้าเก็บรหัสผ่านเป็นข้อความธรรมดา วันหนึ่งมีคนเปิดฐานข้อมูลดู จะเห็นรหัสผ่านของพนักงานทุกคนทันที

เราจะเอามาทำอะไร — ใช้ออกแบบว่าจะผูกสิทธิ์ไว้กับตัวบุคคลหรือกับบทบาท และที่สำคัญกว่าคือ **จะตรวจสิทธิ์ที่จุดไหน** เพราะการซ่อนเมนูกับการปฏิเสธคำขอที่ฝั่งเซิร์ฟเวอร์เป็นคนละเรื่องกัน นอกจากนี้ยังใช้ตัดสินเรื่องการเก็บรหัสผ่านและการเข้ารหัสข้อมูลระหว่างทาง

ถ้าไม่มีหัวข้อนี้ — เกณฑ์การยอมรับที่จะเขียนขึ้นเพื่อทดสอบเรื่องสิทธิ์จะไม่มีฐานทฤษฎี และจะตอบไม่ได้ว่าทำไมการซ่อนเมนูอย่างเดียวจึงไม่พอ

2.1.8 การรวบรวมข้อมูลเพื่อสนับสนุนการตัดสินใจ

ใส่มาทำไม — ปัญหาจากหน้างานข้อ EV-08 คือผู้จัดการต้องรอสรุปยอดและนับสต็อกด้วยมือตอนท้ายวัน จึงมองไม่เห็นสถานการณ์ระหว่างวัน และวัตถุประสงค์ข้อ 1.2.4 กำหนดว่าจะให้ติดตามยอดขาย สต็อก และคิวงานได้แบบเรียลไทม์

ตัวอย่างให้เห็นภาพ — ผู้จัดการเปิดดูยอดขายวันนี้ตอนบ่ายสาม ระบบทำได้สองแบบ

> **แบบที่หนึ่ง** ไล่รวมใบเสร็จของวันนี้ทั้งหมดใหม่ทุกครั้งที่เปิดดู ตัวเลขจะตรงเสมอเพราะคำนวณจากของจริง แต่ถ้าวันไหนขายได้พันใบก็ต้องรวมพันใบทุกครั้งที่กดดู
>
> **แบบที่สอง** เก็บตัวเลขยอดขายสะสมไว้ แล้วบวกเพิ่มทีละใบตอนขาย เปิดดูทีก็อ่านตัวเลขเดียว เร็วมาก **แต่ถ้าวันหนึ่งมีใบไหนบวกพลาดไปใบเดียว ยอดจะผิดถาวรและไม่มีใครรู้** เพราะไม่มีอะไรให้เทียบ

เราจะเอามาทำอะไร — ใช้ตัดสินว่า **ตัวเลขสรุปบนหน้าจอผู้บริหารจะคำนวณเมื่อไร** ต้องรู้ข้อแลกของทั้งสองแบบก่อนจึงจะเลือกได้อย่างมีเหตุผล และเลือกให้เหมาะกับปริมาณรายการจริงของร้านค้าปลีกหนึ่งแห่ง

ถ้าไม่มีหัวข้อนี้ — วัตถุประสงค์ข้อ 1.2.4 จะไม่มีทฤษฎีรองรับ และจะอธิบายไม่ได้ว่าทำไมถึงเลือกวิธีคำนวณแบบที่เลือก

2.1.9 การจัดการสินค้าคงคลัง: จุดสั่งซื้อเพิ่มและการเตือนเชิงรุก

ใส่มาทำไม — ปัญหาจากหน้างานข้อ EV-09 คือการตัดสินใจสั่งซื้อพึ่งประสบการณ์และสัญชาตญาณ ทำให้สินค้าขาดมือหรือค้างสต็อก ปัญหานี้ไม่ใช่ปัญหาทางซอฟต์แวร์ แต่เป็นปัญหาการบริหารสินค้าคงคลัง จึงต้องยืมทฤษฎีจากฝั่งการจัดการมาใช้

ตัวอย่างให้เห็นภาพ — ปูนขายเฉลี่ยวันละ 20 ถุง สั่งจากโรงงานแล้วของมาถึงใช้เวลา 3 วัน แปลว่าระหว่างรอของจะขายไปอีกราว 60 ถุง

> **ถ้ารอให้เหลือ 10 ถุงค่อยสั่ง** ของจะขาดตั้งแต่วันแรกที่รอ ลูกค้าไปซื้อร้านอื่น **ถ้าสั่งตอนเหลือ 200 ถุง** เงินจะจมและปูนค้างนานจนเสื่อมสภาพ **จุดที่เหมาะคือราว 60 ถุงบวกเผื่อไว้อีกส่วนหนึ่ง**

และต่อให้ตั้งจุดสั่งซื้อไว้ถูก **ถ้าตรวจแค่ตอนปิดร้าน** ปูนอาจลดจาก 70 เหลือ 45 ถุงตั้งแต่บ่ายสอง กว่าจะรู้ก็เย็นแล้ว เสียเวลาไปครึ่งวัน

เราจะเอามาทำอะไร — ใช้ตอบว่า **ระบบควรเตือนเมื่อไร** ซึ่งแยกเป็นสองคำถาม คือจะกำหนดระดับที่ต้องเตือนอย่างไร และจะตรวจเป็นรอบตามเวลา หรือตรวจทุกครั้งที่ยอดสินค้าเปลี่ยน

ถ้าไม่มีหัวข้อนี้ — บทจะเหลือแต่ทฤษฎีทางซอฟต์แวร์ล้วน ไม่มีอะไรเชื่อมกับการดำเนินธุรกิจของร้าน ซึ่งเป็นจุดอ่อนสำหรับโครงงานที่มีร้านค้าจริงเป็นกรณีศึกษา

2.2 เทคโนโลยีและเครื่องมือ

กฎของส่วนนี้คือทุกตัวต้องตอบว่า **เลือกเพราะข้อจำกัดอะไร** และ **ไปรับใช้ทฤษฎีข้อไหนใน 2.1** ต่างจาก 2.1 ตรงที่ส่วนนี้เขียนจากมุมหลังตัดสินใจแล้ว จึงอ้างของจริงในระบบได้

2.2.1 Node.js และ Express.js

ใส่มาทำไม — เป็นสภาพแวดล้อมหลักของระบบหลังบ้าน ต้องอธิบายว่าทำไมเลือกตัวนี้

ตรงกับงานเราตรงไหน — รับใช้ 2.1.6 เพราะต้องถือช่องทาง WebSocket ค้างไว้กับหน้าจอคิวหลายเครื่องตลอดวัน สถาปัตยกรรมไม่บล็อกการทำงานของ Node จึงเหมาะ · และรับใช้ 2.1.1 เพราะกลไกมิดเดิลแวร์ของ Express ทำให้เอาส่วนตรวจสิทธิ์ไปคาดหน้าทุกเส้นทางได้ในที่เดียว

ตัวอย่างให้เห็นภาพ — สมมติร้านเปิดหน้าจอคิวไว้ 3 เครื่องตั้งแต่ 8 โมงถึง 6 โมงเย็น ทั้งสามเครื่องเปิดช่องทางค้างไว้กับเซิร์ฟเวอร์ตลอด 10 ชั่วโมงโดยไม่ได้ส่งข้อมูลอะไรเลยเกือบทั้งวัน **ถ้าแต่ละการเชื่อมต่อกินหนึ่งเธรดค้างไว้** เซิร์ฟเวอร์จะเสียทรัพยากรไปกับการรออย่างเดียว แต่รูปแบบของ Node คือรอแบบไม่กินเธรด จึงถือการเชื่อมต่อค้างจำนวนมากได้ · ส่วนมิดเดิลแวร์ ตัวอย่างคือเวลาเพิ่มเส้นทางใหม่สำหรับผู้จัดการ **เราไม่ต้องเขียนโค้ดตรวจสิทธิ์ซ้ำในเส้นทางนั้น** เพราะประกาศตัวตรวจไว้ที่เดียวแล้วครอบทุกเส้นทางที่อยู่ใต้มัน — ซึ่งตรงกับ 2.1.7 ที่บอกว่าการตรวจสิทธิ์ต้องอยู่ฝั่งเซิร์ฟเวอร์ ไม่ใช่แค่ซ่อนเมนู

ถ้าตัดออก — จะไม่มีที่อธิบายว่าทำไมระบบถือการเชื่อมต่อค้างจำนวนมากได้

2.2.2 MySQL และเอนจิน InnoDB

ใส่มาทำไม — เป็นชั้นข้อมูลของระบบ และเป็นหัวข้อที่ต้องอธิบาย **ลึกถึงระดับเอนจิน** ไม่ใช่แค่ชื่อผลิตภัณฑ์

ตรงกับงานเราตรงไหน — สคีมากำหนด `ENGINE=InnoDB` ทุกตาราง · `config/db.js` ตั้งขนาดพูลการเชื่อมต่อไว้สูงกว่าจำนวนผู้ใช้พร้อมกันที่ NFR-06 กำหนด และตั้งค่าให้ตัวเลขทศนิยมไม่ถูกแปลงกลับเป็นทศนิยมลอยตัว

ตัวอย่างให้เห็นภาพ — คำว่า “MySQL” อย่างเดียวยังตอบไม่ได้ว่าระบบกันจ่ายซ้ำได้หรือไม่ เพราะขึ้นกับเอนจินข้างใน

> **เอนจินที่ล็อกทั้งตาราง** เวลาพนักงานคนหนึ่งกำลังหักสต็อกปูน **คนที่กำลังหักสต็อกเหล็กเส้นจะต้องรอไปด้วย** ทั้งที่คนละสินค้ากันโดยสิ้นเชิง และถ้าไฟดับกลางทางก็ย้อนกลับไม่ได้ **InnoDB ล็อกเฉพาะแถว** คนที่หักสต็อกเหล็กเส้นเดินต่อได้ทันที ล็อกจะเกิดก็ต่อเมื่อชนสินค้าตัวเดียวกันจริง ๆ

ส่วนเรื่องทศนิยม ตัวอย่างคือราคาปูน 150.50 บาท คูณ 3 ถุง **ถ้าถูกแปลงเป็นทศนิยมลอยตัวระหว่างทาง อาจได้ 451.49999 แทนที่จะเป็น 451.50** ซึ่งกับเงินเป็นเรื่องยอมไม่ได้ จึงต้องตั้งค่าไว้ไม่ให้แปลง

ถ้าตัดออก — 2.1.2 และ 2.1.3 จะเป็นทฤษฎีลอย เพราะเอนจินที่ไม่รองรับธุรกรรมและล็อกทั้งตาราง ทำให้กลไกทั้งสองใช้ไม่ได้เลย

2.2.3 React, Vite และ React Router

ใส่มาทำไม — เป็นชั้นแสดงผลของผู้ใช้ทั้งสามบทบาท

ตรงกับงานเราตรงไหน — รับใช้ 2.1.6 เพราะหน้าจอคิวต้องเปลี่ยนเองทันทีที่ได้รับข้อความที่ผลักเข้ามา React จัดการส่วนที่ต้องปรับให้เอง · React Router ทำให้สามบทบาทเห็นคนละเส้นทางโดยไม่โหลดหน้าใหม่

ตัวอย่างให้เห็นภาพ — จอคิวที่คลังมีรายการค้างอยู่ 12 ใบ แล้วมีใบที่ 13 ผลักเข้ามา **ถ้าไม่มีเฟรมเวิร์กจัดการให้ เราต้องเขียนเองว่าให้ไปแทรกแถวตรงไหนของหน้าจอ และต้องระวังไม่ให้แถวเดิมกะพริบทั้งหมด** แต่ React ให้เราบอกแค่ว่า “ตอนนี้รายการมี 13 ใบ” แล้วมันไปหาเองว่าต้องแก้เฉพาะส่วนไหนของหน้าจอ · ส่วน React Router ตัวอย่างคือพนักงานคลังกดจากหน้าคิวไปหน้ารายละเอียดใบสั่งซื้อ **หน้าจอเปลี่ยนโดยไม่โหลดใหม่ ช่องทางที่ค้างไว้จึงไม่ขาด** ถ้าโหลดหน้าใหม่ทุกครั้ง การเชื่อมต่อจะต้องตั้งใหม่ตลอด

ถ้าตัดออก — จะอธิบายไม่ได้ว่าหน้าจอเปลี่ยนเองได้อย่างไรเมื่อมีข้อมูลผลักเข้ามา

2.2.4 WebSocket และ Server-Sent Events

ใส่มาทำไม — เป็นตัวจริงที่ทำให้ 2.1.6 เกิดขึ้น ถ้าไม่มีหัวข้อนี้ 2.1 จะมีทฤษฎีการผลักข้อมูลแต่ 2.2 ไม่มีเครื่องมือ

ตรงกับงานเราตรงไหน — ใช้ไลบรารี `ws` ผูกกับเซิร์ฟเวอร์ HTTPS ตัวเดิม จึงได้ช่องทางที่เข้ารหัสโดยไม่ต้องเปิดพอร์ตใหม่ · SSE ไม่ต้องลงไลบรารีเพิ่ม ใช้ความสามารถทยอยส่งข้อมูลของเฟรมเวิร์กที่มีอยู่แล้ว จึงเป็นทางถอยที่แทบไม่มีต้นทุน

ตัวอย่างให้เห็นภาพ — ทั้งสองตัวทำหน้าที่ผลักข้อมูล แต่ไม่เหมือนกัน

> **WebSocket คุยได้สองทาง** พนักงานคลังกดรับงานแล้วส่งกลับผ่านช่องเดิมได้เลย **SSE ส่งได้ทางเดียวจากเซิร์ฟเวอร์มาหาหน้าจอ** ถ้าจะส่งอะไรกลับต้องยิงคำขอปกติแยกไปอีกที

เราจึงใช้ WebSocket เป็นหลัก แล้วให้ SSE เป็นทางถอย **สถานการณ์ที่ทางถอยได้ใช้จริงคือเครือข่ายในร้านหรืออุปกรณ์ระหว่างทางไม่ยอมให้ยกระดับการเชื่อมต่อเป็น WebSocket** ซึ่งเกิดขึ้นได้กับอุปกรณ์เก่า ถ้าไม่มีทางถอย จอคิวจะกลายเป็นจอนิ่งทันที และวัตถุประสงค์ข้อ 1.2.2 จะล้มทั้งข้อ

ถ้าตัดออก — CON-05 ใน SRS ที่บังคับให้ใช้ WebSocket โดยมี SSE เป็นทางสำรอง จะไม่มีที่มา

2.2.5 เครื่องพิมพ์ใบเสร็จความร้อนและชุดคำสั่ง ESC/POS

ใส่มาทำไม — บทที่ 1 §1.3.3 ระบุการพิมพ์ใบเสร็จเป็นขอบเขต และเป็นส่วนที่ต้องคุยกับอุปกรณ์จริง

ตรงกับงานเราตรงไหน — `print-service/lib/escpos.js` เขียนคำสั่งเป็นไบต์เอง ไม่ใช้ไลบรารีสำเร็จรูป · มีคำสั่งเลือกตารางรหัส TIS-620 เพราะข้อความเก็บเป็นยูนิโคดแต่เครื่องพิมพ์ใช้ตารางรหัสไทย · `transport.js` เก็บไบต์ที่ส่งออกไว้ตรวจย้อนหลัง

ตัวอย่างให้เห็นภาพ — เครื่องพิมพ์ใบเสร็จ **ไม่เข้าใจภาษาไทยแบบเดียวกับคอมพิวเตอร์** ในฐานข้อมูลคำว่า “ปูนซีเมนต์” เก็บเป็นยูนิโคด แต่เครื่องพิมพ์มีตารางตัวอักษรของตัวเองและต้องถูกสั่งก่อนว่าให้ใช้ตารางไหน

> **ถ้าไม่ส่งคำสั่งเลือกตาราง TIS-620 ไปก่อน** ใบเสร็จจะพิมพ์ออกมาเป็นตัวอักษรขยะทั้งใบ ทั้งที่ข้อมูลในระบบถูกต้องทุกตัว

ส่วนการเก็บไบต์ที่ส่งออกไว้ ตัวอย่างประโยชน์คือเวลาลูกค้าบอกว่าใบเสร็จพิมพ์ตกบรรทัด **เราเปิดดูได้ว่าระบบส่งอะไรออกไปจริง ๆ** แล้วแยกได้ว่าเป็นความผิดของระบบหรือของเครื่องพิมพ์ — เป็นแนวคิดเดียวกับ 2.1.5 คือเก็บร่องรอยไว้ให้ตรวจย้อนหลังได้

ถ้าตัดออก — A-03 ใน SRS ที่สมมติว่าเครื่องพิมพ์รองรับ TIS-620 จะไม่มีที่อธิบายว่าทำไมเรื่องนี้ถึงสำคัญ

2.2.6 การสร้างรหัส QR สำหรับการชำระเงินตามมาตรฐานพร้อมเพย์

ใส่มาทำไม — บทที่ 1 §1.3.3 ระบุการรับชำระด้วย QR พร้อมเพย์เป็นขอบเขต

ตรงกับงานเราตรงไหน — `backend/lib/promptpay.js` ประกอบข้อมูลตามโครงสร้าง TLV ของ EMVCo เขียนเองทั้งหมด ไม่เรียกบริการภายนอก · ส่วนไลบรารีวาดรูป QR อยู่ฝั่งหน้าเว็บ · **ไม่ต่อ Payment Gateway** พนักงานยืนยันรับเงินเอง

ตัวอย่างให้เห็นภาพ — QR พร้อมเพย์ไม่ใช่รูปภาพวิเศษ แต่เป็น**ข้อความชุดหนึ่งที่ประกอบตามรูปแบบมาตรฐาน** แล้วเอามาวาดเป็นรูป ข้อความนั้นบอกว่าเงินจะเข้าบัญชีไหนและจำนวนเท่าไร เราจึงสร้างเองได้โดยไม่ต้องเรียกบริการของใคร

> **ผลที่ตามมาคือถ้าอินเทอร์เน็ตร้านล่ม ระบบยังออก QR ได้ตามปกติ** เพราะไม่ต้องถามใคร

แต่ต้องพูดข้อเสียด้วยให้ครบ — **ระบบสร้าง QR ได้ แต่ไม่มีทางรู้ว่าเงินเข้าแล้วหรือยัง** ตัวอย่างที่เกิดจริงคือลูกค้ายกมือถือมาโชว์หน้าจอว่าโอนแล้ว **พนักงานต้องเปิดแอปธนาคารของร้านดูเองว่าเงินเข้าจริง** แล้วจึงกดยืนยัน นี่คือที่มาของ A-01 ใน SRS

ถ้าตัดออก — จะไม่มีที่อธิบายข้อจำกัดสำคัญที่สุดของระบบ คือระบบไม่รู้ว่าลูกค้าจ่ายเงินแล้วจริงหรือไม่

2.3 งานวิจัยและระบบที่เกี่ยวข้อง

ส่วนนี้ใส่มาทำไม — เพื่อพิสูจน์ว่าปัญหาที่เราแก้ ยังไม่มีใครแก้ครบในบริบทเดียวกัน ถ้าไม่มีส่วนนี้ โครงงานจะถูกถามว่าซ้ำกับของที่มีอยู่แล้วหรือไม่

วิธีอ่านตารางนี้ — แต่ละงานเลือกมาเพราะ**ใกล้เราคนละด้าน** บางงานใกล้ด้านหน้าที่ บางงานใกล้ด้านบริบทธุรกิจ บางงานใกล้ด้านปัญหา ไม่ได้เลือกมาเพราะชื่อเรื่องคล้ายกัน

งาน	ใส่มาทำไม	ต่างจากเราตรงไหน
ชนะชัย สุรินทร์ และคณะ (2568)	ขอบเขตหน้าที่ใกล้เราที่สุดในกลุ่มที่หาได้	เขาใช้ซอฟต์แวร์สำเร็จรูป ปรับตามกระบวนการเฉพาะของร้านได้จำกัด · และเขาระบุเองว่าควรเพิ่มมาตรการความปลอดภัย ซึ่งเราทำไว้แล้ว
อมรรัตน์ และ วิสุวะ (2568)	**ธุรกิจวัสดุก่อสร้างเหมือนร้านเรา** ยืนยันว่าปัญหาเป็นของทั้งอุตสาหกรรม	เป็นงานวิเคราะห์กระบวนการล้วน ไม่มีระบบซอฟต์แวร์ จึงเสริมกันมากกว่าทับกัน
Tanaman และคณะ (2023)	**ใกล้เราที่สุดในเชิงปัญหา** เขาระบุเองว่าพนักงานหลายคนเขียนลงใบกระดาษเดียวกันแล้วข้อมูลผิด	เขาแก้แค่เปลี่ยนกระดาษเป็นดิจิทัล ไม่ได้คุมความถูกต้องระดับฐานข้อมูล และไม่มีคิวเรียลไทม์
Angellin และคณะ (2023)	ระบุปัญหาการทุจริตในระบบมือ ตรงกับ EV-06 ที่เราเจอ	ไม่มีมาตรการกันจ่ายซ้ำและไม่มีการบันทึกผู้กระทำ
ธนากร และ จิรพัฒน์ (2563)	โครงงานนักศึกษาไทย บริบทร้านค้าปลีกเล็กเหมือนกัน	ไม่มีตัดสต็อกอัตโนมัติ ไม่มีแจ้งเตือน ไม่มีเรียลไทม์ เป็นฐานล่างสุดของการเทียบ
เอก และ ธนชัย (2563)	ใกล้เราที่สุดในกลุ่มโครงงานไทย มีตัดสต็อกและแจ้งเตือนแล้ว	หยุดแค่นั้น ไม่เชื่อมการชำระเงินกับคิว คลัง และไม่กันจ่ายซ้ำ

ตัวอย่างที่ควรยกให้อาจารย์ฟังหนึ่งงาน คือ Tanaman และคณะ (2023) เพราะเป็นงานที่ใกล้ที่สุดและอธิบายเส้นแบ่งได้ชัดที่สุด

> เขาเจอปัญหาเดียวกับเราคือ **พนักงานหลายคนเขียนลงใบกระดาษใบเดียวกัน แล้วตัวเลขผิด** เขาแก้ด้วยการเปลี่ยนจากกระดาษเป็นระบบดิจิทัล ซึ่งช่วยได้จริงในแง่ที่ลายมืออ่านออกและไม่หาย
>
> **แต่ปัญหาไม่ได้หายไปทั้งหมด** เพราะถ้าคนสองคนกดบันทึกรายการเดียวกันพร้อมกันบนระบบดิจิทัล ตัวเลขก็ยังเพี้ยนได้อยู่ดี งานของเราจึงไปต่ออีกชั้นด้วยการคุมความถูกต้องที่ระดับฐานข้อมูลตาม 2.1.2 และ 2.1.3

ตัวอย่างที่แสดงว่าเราไม่ได้เลือกแต่งานที่ด้อยกว่า — งานของ **เอก และ ธนชัย (2563)** ทำได้ถึงขั้นตัดสต็อกอัตโนมัติและแจ้งเตือนสินค้าใกล้หมดแล้ว ซึ่งเท่ากับสองในสี่เรื่องที่เราทำ **เรายอมรับตรง ๆ ว่าสองข้อนี้ไม่ใช่ของใหม่** จุดที่ต่างคือเขาไม่ได้เชื่อมจังหวะชำระเงินหน้าร้านเข้ากับคิวของคลัง และไม่มีการกันจ่ายซ้ำ — การพูดแบบนี้น่าเชื่อถือกว่าการอ้างว่าทุกอย่างของเราใหม่หมด

ข้อสรุปที่ตารางที่ 2.1 พิสูจน์ — ทุกงานทำเรื่องสต็อก บางงานถึงขั้นตัดสต็อกและแจ้งเตือน **แต่ไม่มีงานไหนเลยที่มีคิวเรียลไทม์หรือกันจ่ายซ้ำ**

คำที่ต้องใช้ให้ถูก — เขียนว่า “จากการสืบค้นไม่พบ” ไม่ใช่ “ไม่มีใครทำ” เพราะเข้าไม่ถึงฐานข้อมูลที่ต้องเสียเงิน **ตัวอย่างคำถามที่อาจโดน** คือ “แน่ใจได้ยังไงว่าไม่มีใครทำ” คำตอบที่ปลอดภัยคือบอกว่าค้นจากแหล่งที่เข้าถึงได้แล้วไม่พบ ไม่ใช่ยืนยันว่าไม่มีอยู่จริงในโลก

สรุปหน้าเดียว

ถ้าถูกถามว่า “บทที่ 2 ใส่มาทำไมทั้งบท” ตอบว่า

> ทุกหัวข้อใน 2.1 มาจากปัญหาที่บันทึกไว้ในบทที่ 1 และเป็นความรู้ที่ต้องมีก่อนจึงจะตัดสินใจออกแบบได้ ไม่ได้เลือกจากตำรา · ทุกหัวข้อใน 2.2 บอกได้ว่าเลือกเพราะข้อจำกัดอะไรและไปรับใช้ทฤษฎีข้อไหน · และ 2.3 พิสูจน์ว่ายังไม่พบใครทำครบแบบนี้ในบริบทเดียวกัน